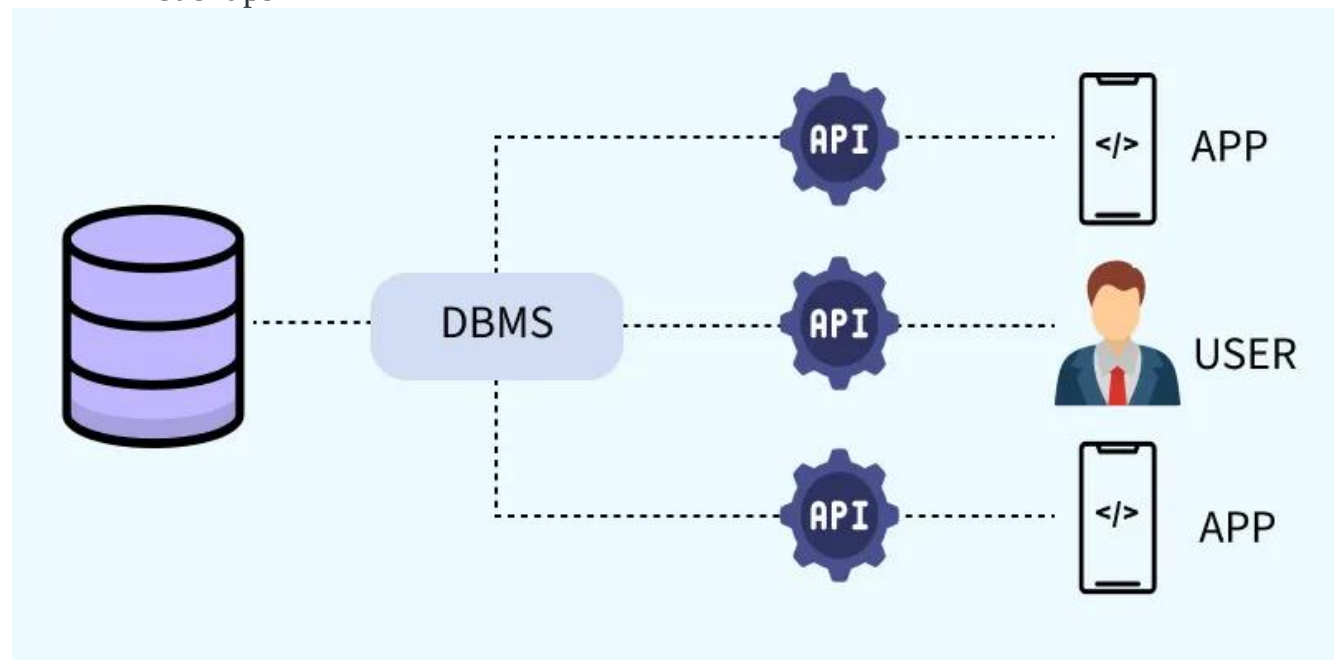


Introduction of DBMS

DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems

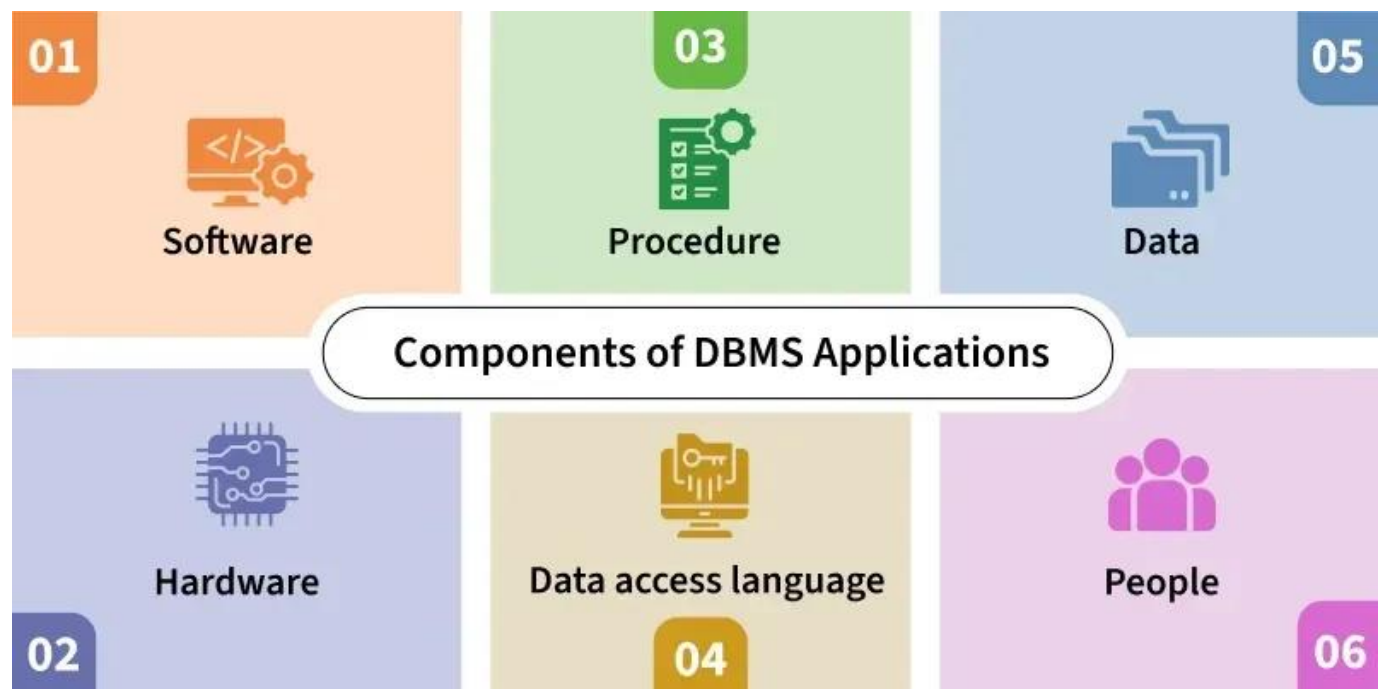
Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Components of DBMS Applications

Any DBMS based applications is made up of six key components that work together to handle data effect Components of DBMS Applications



1. Hardware

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. Software

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. Data

- Raw facts stored in structured or unstructured formats.
- **Operational Data**: Actual user data (e.g., name, age).
- **Metadata**: Data about data (e.g., storage time, size, data type).

- Core reason DBMS exists—to manage and store data efficiently.

4. Procedures

- Instructions and rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

5. Database Access Language

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. People

- Users interacting with DBMS at different levels:
- **Database Administrators (DBA)** – Manage security, performance, user access.
- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Types of DBMS

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. Relational Database Management System (RDBMS)

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL oracle, Microsoft SQL Server and Postgre SQL.

2. NoSQL DBMS

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.
- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Examples: MongoDB, Cassandra, DynamoDB and Redis.

3. Object-Oriented DBMS (OODBMS)

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

Examples: ObjectDB, db4o.

4. Hierarchical Database

- Organizes data in a tree-like structure, where each record (node) has a single parent and have multiple children.
- This model is similar to a file system with folders and subfolders.
- It is efficient for storing data with a clear hierarchy, such as organizational charts or file directories.
- Navigation is fast and predictable due to the fixed structure.
- It lacks flexibility and difficult to restructure or handle complex many-to-many relationships.

Example: IBM Information Management System (IMS).

5. Network Database

- It uses a graph-like model to allow more complex relationships between entities.
- Unlike the hierarchical model, it permits each child to have multiple parents, enabling many-to-many relationships.
- Data is represented using records and sets, where sets define the relationships.
- It is more flexible than the hierarchical model and better suited for applications with complex data linkages.

Example: Integrated Data Store (IDS), TurboIMAGE.

6. Cloud-Based Database

- They are hosted on cloud computing platforms like AWS, Azure or Google Cloud.
- They offer on-demand scalability, high availability, automatic backups and remote accessibility.
- These databases can be relational (SQL) or non-relational (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- They support modern application requirements, including distributed access and real-time analytics.

Example: Amazon RDS (for SQL), MongoDB Atlas (for NoSQL), Google BigQuery.

Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct

role in database management, ensuring efficient storage, retrieval and security of data. The primary database languages include:

Type of DBMS Language				
Data Definition Language	Data Manipulation Language	Data Control Language	Transaction Control Language	Data Query Language
• CREATE • DROP • TRUNCATE • RENAME • ALTER	• INSERT • DELETE • UPDATE	• REVOKE • GRANT	• ROLLBACK • COMMIT	• SELECT

1. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT:** add comments to the data dictionary
- **RENAME:** rename an object

2. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update and delete data.

- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)

- **CALL:** call a PL/SQL or Java subprogram
- **EXPLAIN PLAN:** interpretation of the data access path
- **LOCK TABLE:** concurrency Control

3. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

4. Transaction Control Language (TCL)

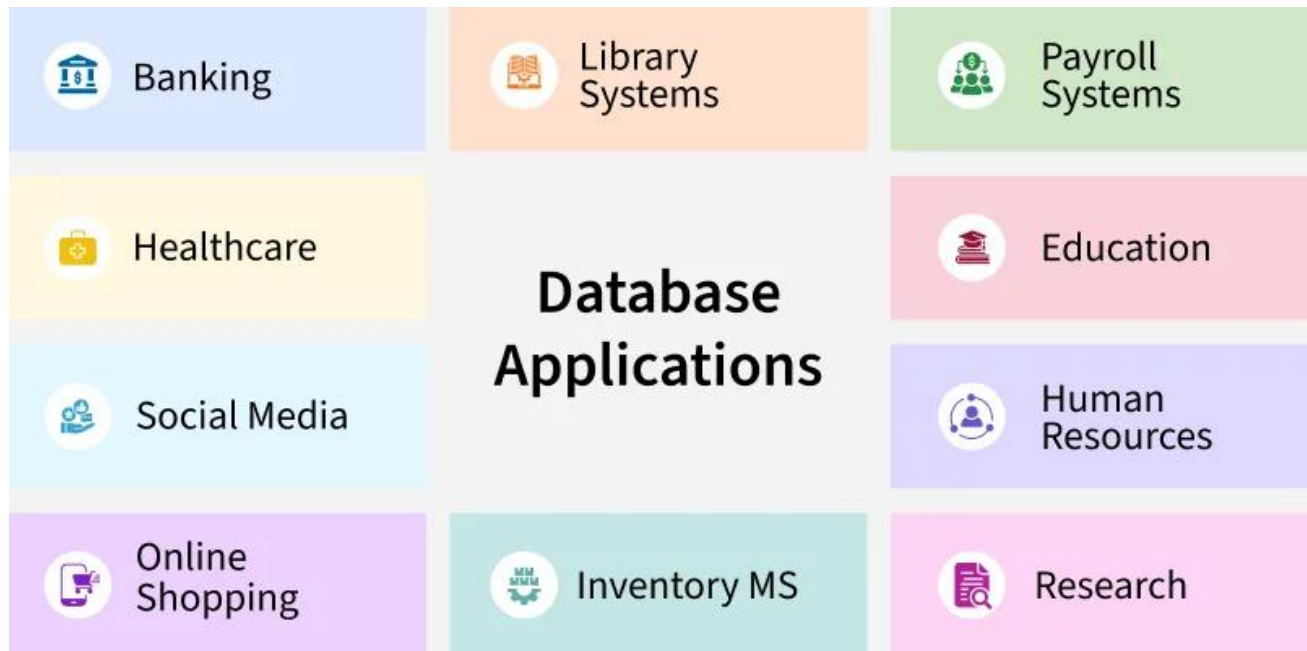
TCL commands oversee transactional data to maintain consistency, reliability and atomicity.

- **ROLLBACK:** Undoes changes made during a transaction.
- **COMMIT:** Saves all changes made during a transaction.
- **SAVEPOINT:** Sets a point within a transaction to which one can later roll back.

5. Data Query Language (DQL)

DQL is a subset of SQL used to retrieve data from a database without modifying it. Its main command is SELECT, which allows users to fetch specific information based on their requirements.

Applications of DBMS



- **Banking:** Manages accounts and transactions.
- **E-commerce:** Tracks products, orders, and customers.
- **Healthcare:** Stores patient records and diagnoses.
- **Education:** Handles student grades and schedules.
- **Social Media:** Manages user profiles and interactions.
- **Data Science:** Supports analytics and predictions.

Data Dictionary

Think of a Data Dictionary as a "Database Encyclopedia"

It doesn't store the *actual data* (like student names or grades). Instead, it stores **information about the data** — the *metadata*.

What It Contains

- **Table names** → e.g., *Students*, *Courses*
 - **Column names & types** → *Student_ID* (*integer*), *Student_Name* (*varchar*)
 - **Constraints** → Primary keys, foreign keys, unique rules
 - **Relationships** → How tables connect (e.g., *Students* linked to *Courses* via *Enrollments*)
 - **Permissions** → Who can access or modify the data
-

Simple Analogy

Imagine a **library catalog**:

- The catalog doesn't hold the books themselves.
- It holds **information about the books**: title, author, subject, shelf location.
- That way, you know what's in the library and how to find it.

A **data dictionary** works the same way for a database.

Example: Student Database Data Dictionary

Table	Column	Data Type	Constraint	Description
Students	Student_ID	Integer	Primary Key	Unique ID for each student
Students	Student_Name	Varchar(50)	Not Null	Full name of student
Students	Student_City	Varchar(30)	—	City of residence
Courses	Course_ID	Integer	Primary Key	Unique ID for each course
Enrollments	Student_ID	Integer	Foreign Key	Links to Students table
Enrollments	Course_ID	Integer	Foreign Key	Links to Courses table

Why It's Important

- **Clarity** → Everyone knows what each column means.
 - **Consistency** → Prevents confusion (e.g., “Is *ID* a student or course ID?”).
 - **Governance** → Helps with auditing, compliance, and security.
 - **Collaboration** → Developers, analysts, and managers all use the same definitions.
-

👉 In short:

A **data dictionary** = **metadata catalog**. It’s the *map* of your database, not the data itself.

What is Metadata?

Metadata is simply “**data about data.**” It describes the structure, properties, and context of actual data, rather than being the data itself.

Key Characteristics

- **Definition:** Metadata provides information that helps you understand, organize, and manage data.
 - **Purpose:** It makes data easier to find, interpret, and use correctly.
 - **Stored in:** Data dictionaries, catalogs, or system tables in a DBMS.
-

Types of Metadata

Type	Description	Example
Structural Metadata	Describes how data is organized.	Table names, column types, relationships.
Descriptive Metadata	Provides context about the data.	Author, creation date, file size.
Administrative Metadata	Manages usage and access.	Permissions, ownership, last modified date.

Example

Imagine a **photo file**:

- **Actual Data** → The image pixels (the photo itself).

- **Metadata** → File name, size, resolution, date taken, camera type.

In databases:

- **Actual Data** → Student names, grades, addresses.
- **Metadata** → Column definitions (e.g., *Student_Name* is *VARCHAR(50)*), constraints (Primary Key on *Student_ID*), relationships (linked to *Courses*).

Everyday Analogy

Think of metadata like the **label on a food package**:

- The food inside = actual data.
- The label (ingredients, nutrition facts, expiration date) = metadata.

Here's a **side-by-side visual table** to make the difference between **data** and **metadata** crystal clear:

Data vs Metadata

Aspect	Data (Actual Content)	Metadata (Data About Data)
Photo File	The image pixels (the photo itself).	File name, size, resolution, date taken, camera type.
Book	The text of the book (story, chapters).	Title, author, ISBN, publication date, genre.
Database	Student names, grades, addresses.	Table names, column types, constraints (Primary Key, Foreign Key), relationships.
Music File	The audio track (song itself).	Artist, album, duration, file format, release year.

Easy Analogy

- **Data** = The *thing itself* (photo, book, song, student record).
 - **Metadata** = The *label or description* that explains what the thing is, how it's structured, and how to use it.
-

Facebook primarily uses **MySQL** as the foundation of its database system, but it has built a highly complex **polyglot persistence architecture** around it to handle billions of daily operations.

Breakdown of Facebook’s Database Ecosystem

Technology	Role at Facebook	Why It’s Used
MySQL (sharded)	Core relational database for storing social data (users, posts, relationships).	Reliable, mature, and flexible. Facebook customizes it heavily for scalability.
TAO (custom NoSQL layer)	A graph API built on top of sharded MySQL.	Provides fast access to social graph data (friends, likes, comments) with low latency.
Apache Cassandra	Handles large-scale user engagement data.	Highly scalable, distributed, fault-tolerant.
Apache HBase	Used for analytics and massive data storage.	Efficient for handling billions of rows and real-time queries.
RocksDB (Facebook-built)	Embedded key-value store for high-performance workloads.	Optimized for fast storage and retrieval.

Why Facebook Uses This Setup

- **Scalability:** MySQL alone couldn’t handle billions of users, so Facebook built TAO and sharded MySQL clusters to distribute the load.
- **Performance:** Different workloads (social graph queries, analytics, engagement tracking) require specialized databases.
- **Flexibility:** A polyglot approach lets Facebook pick the best tool for each type of data.
- **Reliability:** MySQL’s maturity and Facebook’s custom optimizations make it a stable backbone.

👉 In short:

- **MySQL** is the foundation.
- **TAO + sharding** makes it scalable.
- **Other databases (Cassandra, HBase, RocksDB)** handle specialized tasks.

Here’s a **color-coded decision matrix** comparing why Facebook uses each database in its stack. This highlights trade-offs across key dimensions like scalability, latency, consistency, and flexibility:

Facebook Database Decision Matrix

Database	Scalability	Latency	Consistency	Flexibility	Use Case
MySQL	□ High	□ Moderate	□ Strong	□ Moderate	Core relational data (users, posts)
TAO (on MySQL)	□ Very High	□ Low	□ Eventual	□ High	Social graph queries (likes, friends)
Cassandra	□ Very High	□ Low	● Weak	□ High	Engagement, messaging, time-series data
HBase	□ Very High	□ Moderate	□ Strong	□ Moderate	Analytics, massive data storage
RocksDB	□ High	□ Very Low	□ Strong	□ High	Embedded key-value store, fast access

Color Legend

- □ Excellent
- □ Moderate
- ● Limited

This matrix shows how Facebook strategically balances **speed, scale, and consistency** by using each database where it performs best. TAO adds speed to MySQL, Cassandra scales horizontally, HBase handles analytics, and RocksDB delivers ultra-fast access.

Understanding Data Abstraction

Data abstraction is a **core concept in computer science and software engineering** that helps manage complexity by hiding unnecessary details and exposing only the essential features.

Key Points

- **Definition:** Data abstraction is the process of providing only relevant information to the user while concealing the background details of how things work.
- **Purpose:** It simplifies interaction with systems, making them easier to use, more secure, and more adaptable.
- **Analogy:** Driving a car is a classic example—you use the steering wheel, accelerator, and brakes without needing to understand how the engine, transmission, or braking system works internally.

Levels of Abstraction

1. **Physical Level** – How data is stored physically (e.g., on disks).
2. **Logical Level** – What data is stored and the relationships among them.
3. **View Level** – How data is presented to users (e.g., through interfaces or APIs).

Benefits

- **Simplifies complexity:** Developers can focus on high-level design without worrying about low-level details.
- **Improves security:** Sensitive implementation details remain hidden.
- **Enhances flexibility:** Systems can evolve internally without affecting how users interact with them.

Example in Programming

- **Abstract Data Types (ADTs):** Like stacks, queues, or lists, which define operations (push, pop, enqueue, dequeue) without revealing how they are implemented.

Here's a clear **visual comparison table** of the three levels of data abstraction, with examples to make the concept easy to grasp:

Levels of Data Abstraction in DBMS

Level	What It Shows	Hidden Details	Example
Physical Level	How data is physically stored (files, disks, indexes).	Hardware details, storage structures, compression.	Data stored as blocks on a hard drive.
Logical Level	What data is stored and relationships among them.	Physical storage details.	A database schema showing tables like <i>Students</i> and <i>Courses</i> with relationships.
View Level	How data is presented to users.	Logical and physical details.	A student sees only their grades via a web portal, not the entire database.

Quick Way to Remember

- **Physical** → *Where and how data lives*
- **Logical** → *What data exists and how it connects*
- **View** → *What the user sees*

This layered approach ensures **simplicity, security, and flexibility**—users interact only with what they need, while developers and systems handle the complexity behind the scenes.

Data Abstraction and Data Independence

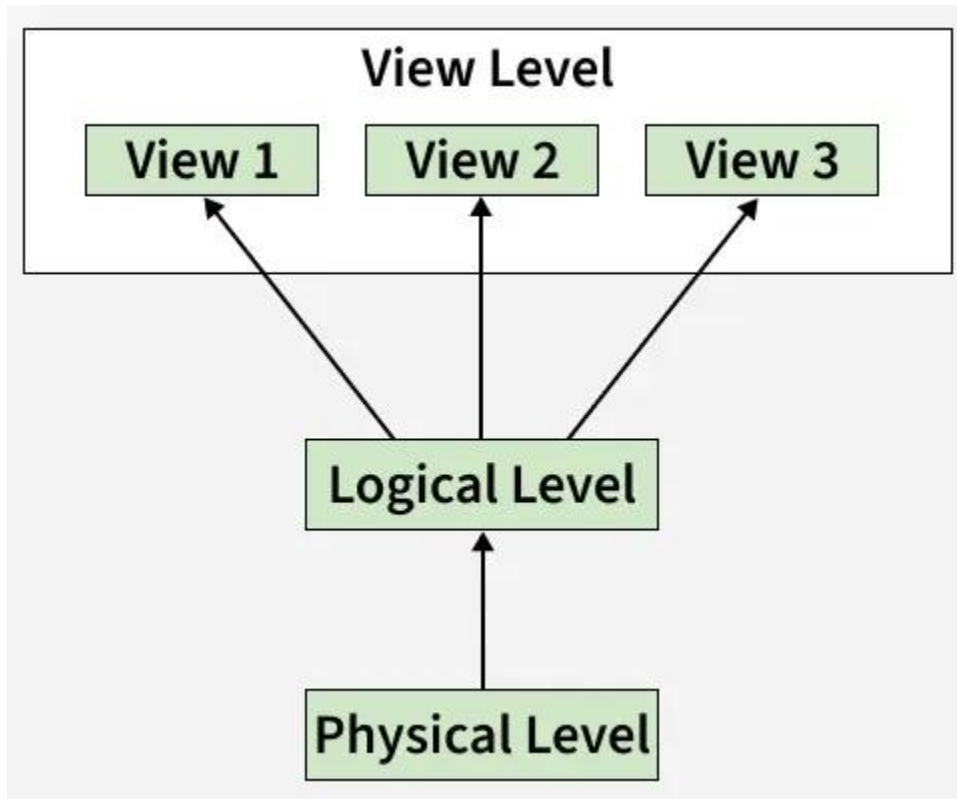
A Database Management System (DBMS) is designed to efficiently store, organize and retrieve large volumes of data. However, because of the complexity of databases, it is essential to simplify how users interact with the system. To achieve this,

DBMS uses a concept called data abstraction, which hides unnecessary details and presents only relevant information to different users.

This simplification also enables data independence, ensuring that changes in one level of the database do not affect other levels.

Data Abstraction

The main goal of data abstraction is to simplify database management and achieve data independence. It allows users to work with data without worrying about how it is stored or maintained internally, reducing complexity and improving efficiency. There are mainly three levels of data abstraction in a DBMS:



Data Abstraction

Physical or Internal Level

Logical or Conceptual Level

View or External Level

1. **Physical or Internal Level**

This is the lowest level of abstraction that deals with how data is physically stored in memory.

It defines file organization techniques (like hashing or B+ trees) and access methods (such as sequential or random access).

Details such as block size, memory allocation and storage paths are managed at this level.

These physical details are hidden from the end users.

Example: When storing employee details, the user doesn't need to know how many memory blocks are used or where the data is stored on disk.

2. Logical or Conceptual Level

This level describes what data is stored in the database and the relationships among those data.

It represents the entire database using simple logical structures like tables, attributes and relationships.

The logical level provides physical data independence, meaning that changes at the physical level do not affect this level.

Database administrators primarily work at this level to define schemas and relationships.

Example: At this level, employee details are represented as tables with fields such as Emp_ID, Emp_Name and Manager_ID.

3. View or External Level

This is the highest level of abstraction, where users interact directly with the database.

It provides different views of the same data for different users, ensuring security and simplicity.

Users access data through Graphical User Interfaces (GUI) or Command-Line Interfaces (CLI) without dealing with the underlying complexity.

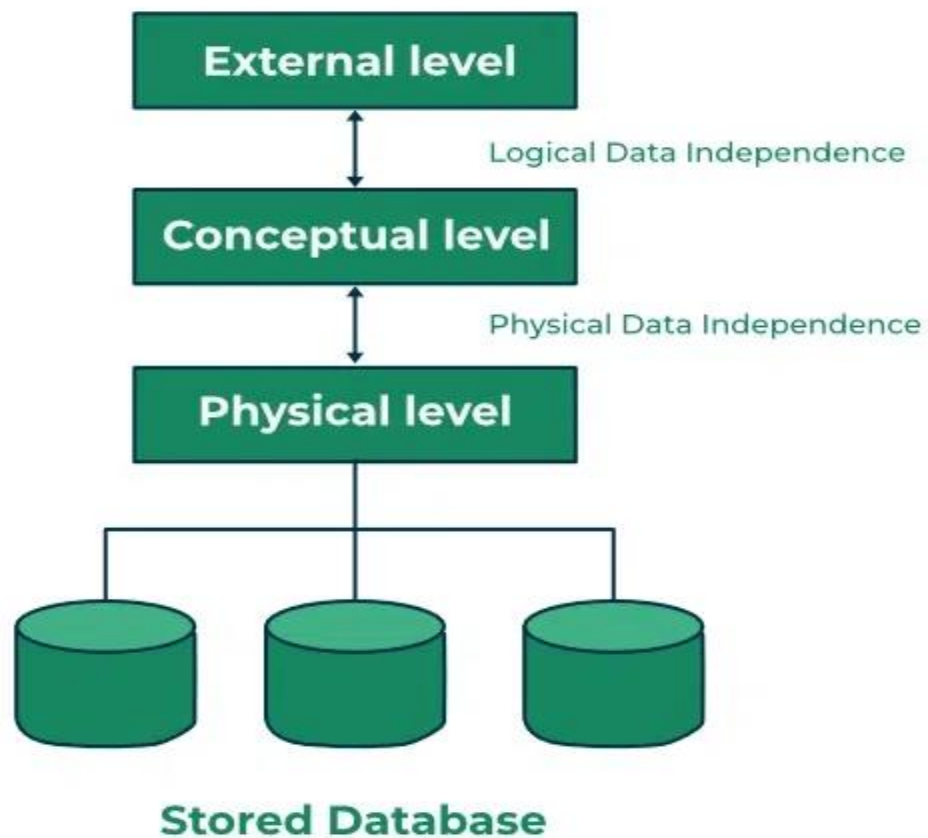
Example: A user may view only the Emp_Name and Department columns, while other details remain hidden.

Data Independence

Data Independence is the property that allows changes in the database schema at one level without requiring changes at the next higher level. It helps separate data

from the programs that use it, thereby improving flexibility and minimizing maintenance effort.

Data Independence in DBMS



Example of Data Independence

Consider a database storing customer information:

Physical level: Stores data in files and manages memory blocks.

Logical level: Represents customers as a table with attributes like Customer_ID, Name and Phone.

View level: A user may see only Name and Phone fields.

Note: If the storage method changes (physical level) or a new column Email is added (logical level), the user view remains unaffected - demonstrating data independence.

There are two types of data independence:

1. Physical Data Independence

It refers to the ability to change the physical storage of data without affecting the logical schema.

Changes like altering indexes, switching file organization methods or upgrading storage devices do not affect the conceptual structure of the database.

Examples:

Changing from sequential file organization to hashing.

Using a new storage device or modifying access paths.

Benefit: It allows performance optimizations at the physical level without modifying higher-level structures.

2. Logical Data Independence

It refers to the ability to modify the logical schema (tables, attributes, relationships) without affecting the user views or application programs.

Users can continue to access data without changes in their queries even if new fields are added or relationships modified in the conceptual schema.

Examples:

Adding or deleting attributes in a table.

Changing the relationship between entities.

Benefit: , It allows the database to evolve without disturbing existing applications.